

OmniCue Blueprint

Real-Time Meeting Context-Matching Prototype — Project & Execution Roadmap

The Core Value Proposition

OmniCue is an autonomous, silent meeting companion powered by Gemini 3 and Google Cloud Agent Builder. It eliminates the friction of post-meeting lookups and "let me check and get back to you" delays by analyzing live conversational intent, querying structural corporate data vaults via the Elastic Model Context Protocol (MCP), and instantly surfacing key documentation right onto the user's workspace.

1. The End-to-End Conceptual Flow

To operate smoothly without introducing conversational latency, the pipeline runs as a synchronized asynchronous loop across five discrete architectural components:

1. **Capture Block:** The client-side dashboard tracks the system microphone input natively via the browser's *Web Speech API*, chunking running audio text streams dynamically.
2. **Transmission Layer:** A lightweight WebSockets server buffers raw streaming characters and marshals structured text packets into the backend every time a user finishes a natural phrase.
3. **Orchestration Gateway (The Brain):** The *Google Cloud Agent Builder* (housing Gemini 3 Flash) ingests the streaming block. If semantic evaluation triggers an information-gathering intent, it translates the text context into an optimized query.
4. **Superpower Integration Link:** The agent acts on the intent by invoking the deployed *Elasticsearch MCP Server* over a secure HTTP transport interface.
5. **Data Resolution Layer:** Elasticsearch executes a hybrid semantic and lexical search across indexed internal documents, passing the matching metadata snippet back through the WebSocket directly to a sliding UI overlay box on screen.

2. Shared Integration API Contract

To enable decoupled, parallel work execution between the frontend client and the backend core system, both subsystems will bind explicitly to the following fixed WebSocket communication contracts:

A. Speech Ingestion Event (Client → Server)

Fired by the frontend microphone listener every time a coherent block of spoken dialogue passes a 1.5-second pause barrier:

```
// Event Channel Identifier: "meeting_speech_stream"
{
  "session_id": "omnicue-hack-001",
  "text": "We need to verify the code freeze date for the Q2 launch roadmap."
}
```

B. Context Match Delivery Event (Server → Client)

Fired by the backend integration service immediately after Gemini 3 orchestrates a successful retrieval from the Elastic cluster index:

```
// Event Channel Identifier: "context_match_found"
{
  "title": "Q2_Release_Plan.md",
  "snippet": "Code Freeze is officially scheduled for June 15, 2026. No further main  
branch pushes are allowed.",
  "url": "https://confluence.company.com/pages/viewpage.action?pageId=89712"
}
```

3. Complete Step-by-Step Prototype Implementation Roadmap

Phase 1: Setting up the Elastic Data Layer

Establish a test database deployment using an Elastic Cloud free tier cluster instance. Populate the cluster index with 5 to 10 highly realistic markdown document objects that simulate a typical corporate workspace environment (e.g., product release roadmaps, infrastructure configuration guides, and client SLA contracts). Ensure that one document contains the explicit string parameters used to validate the pitch script demo.

Phase 2: Deploying the Elastic MCP Server

Package the official `elasticsearch-mcp-server` image within a Docker runtime container and deploy it publicly to **Google Cloud Run**. Configure cluster authorization mappings through explicit environmental variables: `ES_URL`, `ELASTICSEARCH_USERNAME`, and `ELASTICSEARCH_PASSWORD`. Validate the external connectivity link using an MCP Inspector utility to verify that the core search mechanism behaves as an open Streamable-HTTP interface.

Phase 3: Configuring Google Cloud Agent Builder

Instantiate a smart developer agent inside Google Cloud Agent Builder backed directly by the **Gemini 3 Flash** foundation model. Under the "Tools" sub-menu, establish a new integration utility mapped to the custom **MCP** plugin option and paste the target Cloud Run execution link. The engine will read the schema definition automatically and acquire the data matching capabilities. Program the following behavior constraint model directly into the primary Agent system prompt configuration block:

You are a passive enterprise listening companion sitting in a live call.

1. Never speak back directly unless called explicitly by your name.
2. Continually parse the text inputs. If they mention explicit software plans, agreements, or technical settings, translate that intent into an optimized Elasticsearch Query DSL string.
3. Call the Elastic MCP 'search' tool and return the title, snippet, and url parameters within a 'document_found' object map.

Phase 4: Creating the Client Audio Streamer

Construct a modern dashboard UI featuring an active voice hook. Instantiate the browser's webkitSpeechRecognition utility with continuous data collection toggled on (recognition.continuous = true). Integrate a structural script buffer that watches the stream real-time and executes the WebSocket transmitter only after detecting distinct speech pauses, avoiding data floods.

Phase 5: Finalizing the Integration Link

Assemble the Node.js middleware wrapper service using Express and Socket.io. Write the logic loops that catch incoming client audio chunks, dispatch them directly to the Google Cloud Agent Builder session runtime client via the detectIntent() framework, map any resultant payload returns, and relay the document metadata down to the active web client in real time.

4. The Hackathon Task Split

To maximize velocity over the course of the challenge, development duties are partitioned cleanly into separate, parallel execution tracks:

Work Track & Ownership	Assigned Operational Deliverables
MY TASKS Data & AI Brain Architecture	• Configure and seed the mock corporate database inside Elastic Cloud. • Expose the Elasticsearch index by deploying the MCP Server onto Google Cloud Run. • Provision the core runtime model workspace inside Google Cloud Agent Builder using Gemini 3 Flash. • Author system prompt criteria to drive context matching and clean query formulation. • Build the Node.js integration server that acts as the messaging intermediary.

Work Track & Ownership	Assigned Operational Deliverables
TEAMMATE TASKS Audio Capture & Interface UI	• Build the web-based meeting application workspace interface. • Integrate the browser-native Web Speech API script handler. • Write the 1.5-second text chunk debouncing algorithm to bundle raw verbal audio lines. • Establish the frontend WebSocket network interface to stream textual transcripts. • Design and polish a floating overlay component that catches data returns and glides elegantly onto the screen.

5. Enterprise Financial Feasibility Framework

For the hackathon submission pitch, the underlying architecture maps to an exceptionally scalable and lean cost profile. Scaled across a 1,000-employee company storing a comprehensive 10-year archive (2.5 million files / 500 GB indexing footprint), the ongoing operational costs break down as follows:

Infrastructure Layer	Monthly Operational Cost	Capacity Driver Mapping
Real-Time Speech-to-Text Ingestion	\$3,000.00	10,000 unique blended company call hours
Gemini 3 Flash LLM API Volume	\$230.00	Continuous context processing (410M tokens)
Memory-Optimized Elasticsearch Cluster	\$1,000.00	500 GB storage footprint / 22.5 GB Vectors in RAM
Cloud Orchestration & State Tracking	\$500.00	WebSocket pipeline routers and event gateways
Total Monthly Enterprise Commitment	\$4,730.00	Blended Rate: Just \$4.73 / User / Mo.

This metrics engine underpins a clear path to production readiness, presenting judges with a highly optimized, capital-efficient deployment story built natively around official ecosystem partner tracks.